

JavaScript Loops: Deep Dive, Practice & Application

Deep Dive Comparison Guide:

Loop Type	Syntax Example	Best Use Cases	Strengths	Pitfalls
for	<code>for (let i = 0; i < 5; i++) {}</code>	Known number of iterations	Simple, concise	Off-by-one errors, infinite loops if increment or condition is wrong
while	<code>while (condition) {}</code>	Looping until a condition is no longer true	Great when the end condition isn't known	Risk of infinite loops if condition never becomes false
do...while	<code>do {} while (condition);</code>	Execute at least once regardless of condition	Guarantees one execution	Same infinite loop risk as while
for...in	<code>for (let key in object) {}</code>	Looping through object properties	Useful for objects	Not ideal for arrays (may include prototype properties)
for...of	<code>for (let value of array) {}</code>	Looping through iterable values like arrays, strings	Cleaner than for when just values are needed	Doesn't work with non-iterables (e.g., plain objects)

2. Practice Problems (Loops + Conditionals)

1. FizzBuzz (1 to 30)

```
for (let i = 1; i <= 30; i++) {  
  if (i % 3 === 0 && i % 5 === 0) console.log("FizzBuzz");  
  else if (i % 3 === 0) console.log("Fizz");  
  else if (i % 5 === 0) console.log("Buzz");  
  else console.log(i);  
}
```

output:

1

2

Fizz

4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
...

2. First 10 Prime Numbers

```
let count = 0;
let num = 2;
while (count < 10) {
  let isPrime = true;
  for (let i = 2; i <= Math.sqrt(num); i++) {
    if (num % i === 0) {
      isPrime = false;
      break;
    }
  }
  if (isPrime) {
    console.log("Prime:", num);
    count++;
  }
  num++;
}
```

output:

Prime: 2

Prime: 3

Prime: 5

Prime: 7

Prime: 11

Prime: 13

Prime: 17

Prime: 19

Prime: 23

Prime: 29

3. Star Triangle Pattern

```
let rows = 5;
for (let i = 1; i <= rows; i++) {
  let stars = "";
  for (let j = 1; j <= i; j++) {
    stars += "*";
  }
  console.log(stars);
}
```

output:

*

**

4. Reverse Countdown with do...while

```
let countdown = 5;
```

```
do {
```

```
console.log("Countdown:", countdown);  
countdown--;  
} while (countdown >= 0);  
output:Countdown: 5  
Countdown: 4  
Countdown: 3  
Countdown: 2  
Countdown: 1  
Countdown: 0
```

5. Object Loop with for...in

```
const car = { brand: "Toyota", year: 2020, color: "red" };  
for (let key in car) {  
  console.log(`${key}: ${car[key]}`);  
}  
output:brand: Toyota  
year: 2020  
color: red
```

6. Array Loop with for...of

```
const cities = ["Paris", "London", "Tokyo"];  
for (let city of cities) {  
  console.log("City:", city);  
}  
output:  
City: Paris  
City: London  
City: Tokyo
```

7. Sum of Even Numbers (1–20)

```
let sum = 0;
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) sum += i;
}
console.log("Sum:", sum);
```

output:

```
let sum = 0;
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) sum += i;
}
console.log("Sum:", sum);
```

output:

Sum: 110

8. Multiplication Table (5x5)

```
for (let i = 1; i <= 5; i++) {
  let row = "";
  for (let j = 1; j <= 5; j++) {
    row += (i * j).toString().padStart(4);
  }
  console.log(row);
}
```

output:

```
for (let i = 1; i <= 5; i++) {
  let row = "";
  for (let j = 1; j <= 5; j++) {
    row += (i * j).toString().padStart(4);
  }
  console.log(row);
}
```

9. Factorial of 5

```
let fact = 1;

let n = 5;

for (let i = 1; i <= n; i++) {
    fact *= i;
}

console.log(`Factorial of ${n} is`, fact);

output: Factorial of 5 is 120
```

10. Print 1–50 (skip multiples of 7)

```
for (let i = 1; i <= 50; i++) {
    if (i % 7 === 0) continue;
    console.log(i);
}

output:

Prints numbers 1 to 50 excluding multiples of 7 (i.e., 7, 14, 21, ... 49).
```

3.JavaScript Function: Multiplication Table Generator

```
function printMultiplicationTable(n) {
    for (let i = 1; i <= n; i++) {
        let row = "";
        for (let j = 1; j <= n; j++) {
            row += (i * j).toString().padStart(4); // Format for alignment
        }
        console.log(row);
    }
}
```

Explanation:

- for (let i = 1; i <= n; i++): Outer loop controls the rows.
- for (let j = 1; j <= n; j++): Inner loop controls the columns.
- i * j: Multiplies current row value by current column value.
- padStart(4): Adds spaces for proper alignment.

- `console.log(row)`: Prints each row as a line.

Example Output for n = 5:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25